

CSAPP一至三章

@南宫萧樱

数据部分：

字和机器位数相关，32位机器就是4字节一个字，表示所能表示地址空间大小。也就是通用寄存器的位数。

内存每次只能存储一个字节，8位。

0x12345678 的小端存储示例：

地址	数据 (byte)
0x1000	0x78
0x1001	0x56
0x1002	0x34
0x1003	0x12

基本数据类型的字节数 (x86/64)：

char:1 short:2 int:4 long:8 long long:8 float:4 double:8 long double:10/16

逻辑右移是直接右移，算术右移是左边补充最高位 ($1001 \gg 2 == 1110$)

补码怎么求：写出绝对值的二进制→按位取反→+1

有符号整数： $T_{min}=(100\dots0)_2$, $T_{max}=(011\dots1)_2$, $-1=(111\dots1)_2$

无符号数和有符号数相互转换之后会出现**解释方式**改变，导致有些数（补码位）对应时要加减最高位（就是最高位当什么解读的区别）。

隐式转换：有符号和无符号混用时，有符号数会隐式转换成无符号数。这不叫改变位模式而是叫重新解读！

符号扩展：把最高位（符号位）库库往高位复制。

无符号截断：就是对 2^k 取模

有符号截断：取模之后还要按照补码解读

加法：CPU不知道自己在处理符号数还是无符号数

情况	是否可能溢出	示例	备注
正 + 正	可能溢出	$3 + 5 = -8$ (错) (假例子不要在意)	超出正数最大值时出错
负 + 负	可能溢出	$-5 + -5 = 6$ (错)	超出负数最小值时出错
正 + 负	不会溢出	$5 + (-2) = 3$	安全
负 + 正	不会溢出	$-3 + 2 = -1$	安全

无符号数乘法直接丢弃高位，保留低 w 位；有符号数也是保留低 w 位。（实际上乘法的时候会扩展）

我们希望有符号数向右算数位移，向零舍入，但是事实上不会，所以我们要用如下操作：

负数时，若要计算 $\lceil (x/2^k) \rceil$ ，可以计算 $\lfloor ((x + 2^k - 1)/2^k) \rfloor$ 也就是 $(x+(1<<k)-1)>>k$

`size_t` 定义为长度为计算机程序相同字长的无符号数

IEEE754浮点数规则：

$$(-1)^s \times 1.f \times 2^{exp-bias}$$

名称	单精度 (float)	双精度 (double)
符号位 s	1 bit	1 bit
阶码 exp	8 bits	11 bits
尾数 f	23 bits	52 bits

单精度：

规格化数： $exp \neq 000\dots 0$ and $exp \neq 111\dots 1$

$$exp = E + bias$$

$$bias = 2^{k-1} - 1$$

k 是阶码位数

单精度：

$$exp : 1, \dots, 255 - 1, E = -126, \dots, 127$$

双精度：

$$exp : 1, \dots, 2047 - 1, E = -1022, \dots, 1023$$

尾数编码隐含先导数值1.

$$f : 0000\dots 0 - > 1111\dots 1$$

注意 f 在添加零的过程中是在右边添加0，不是像整数那样在左边添加。

特殊值： $exp = 111\dots 1$

1. $exp = 111\dots 1, f = 000\dots 0$, 无穷

2. $exp = 111\dots 1, f \neq 000\dots 0$, NaN

非规格化数： $exp = 000\dots 0$

$$E = 1 - bias = -126 \text{ or } -1022$$

尾数： 0.xxxxxxx

1. $f = 000\dots 0$, 表示0

2. $f \neq 000\dots 0$, 最接近0.0的数，间隔均匀

浮点数0有两种表达方式（正负零），但是在比较时相等（`==`）

NaN 跟任何值比较返回 false 除了 NaN!=NaN

浮点数乘法：

$S=S1 \wedge S2$ (异或) $M=M1 * M2$ (尾数相乘) $E=E1+E2$ (指数相加)

如果 $M \geq 2$ 则 M 右移一位, E 加 1, 将 M 舍入以符合小数部分精度要求。使用“最接近偶数”舍入规则 (round to nearest, ties to even)

浮点数加法：

$$A = (-1)^{s_1} \times 1.f_1 \times 2^{e_1}$$

$$B = (-1)^{s_2} \times 1.f_2 \times 2^{e_2}$$

例如：

A: 1.101×2^5

B: $1.011 \times 2^3 \rightarrow 0.01011 \times 2^5$ (右移2位使得指数向较大者对齐)

小数部分：

$$1.101 + 0.01011 = 1.11111$$

重新规格化, 舍入。

浮点数与整型转换：

double/float->int: 最近的整数, 向零靠拢, 无定义时设置成TMin

注意 int 转 float 会有精度损失, double 不会, 它宽。

浮点数溢出不会改变符号位

浮点数加法不满足结合律, 可能大数吃小数, 乘法不满足分配律

汇编代码部分：

后缀	含义	大小	示例
b	byte	8 位	movb
w	word	16 位	movw
l	long	32 位	movl
q	quad word	64 位	movq, testq

testq %rax, %rax 表示对 **64位寄存器** %rax 做操作, 如果你写 testl %eax, %eax, 那就是只对 **低32位** %eax 做操作。

程序计数器 (通常称为 "PC", 在 **x86-64** 中用 %rip 表示) 给出将要执行的下一条指令在内存中的地址。

常用寄存器：

%rax	%eax	%ax	%al	累加器：用于算术运算, 函数返回值
%rbx	%ebx	%bx	%bl	

<code>%rax</code>	<code>%eax</code>	<code>%ax</code>	<code>%al</code>	累加器：用于算术运算，函数返回值
<code>%rcx</code>	<code>%ecx</code>	<code>%cx</code>	<code>%cl</code>	第四个参数
<code>%rdx</code>	<code>%edx</code>	<code>%dx</code>	<code>%dl</code>	第三个参数
<code>%rsi</code>	<code>%esi</code>	<code>%si</code>	<code>%sil</code>	第二个参数
<code>%rdi</code>	<code>%edi</code>	<code>%di</code>	<code>%dil</code>	第一个参数
<code>%rbp</code>	<code>%ebp</code>	<code>%bp</code>	<code>%bpl</code>	栈帧指针寄存器（Base Pointer）
<code>%rsp</code>	<code>%esp</code>	<code>%sp</code>	<code>%spl</code>	栈指针
<code>%rip</code>				指令寄存器（下一条指令地址）
<code>%xmm0~%xmm15</code>				浮点数

不能直接从内存到内存传输数据

当你要把一个**超过 32 位的立即数**加载到寄存器（如 `%rax`）中时，用 `movabsq`：

```
movabsq $0x1234567890abcdef, %rax
```

这条指令会把 64 位的立即数 `0x1234567890abcdef` 直接放进 `%rax`，这是标准 `movq` 指令做不到的（因为普通 `movq` 在加载立即数时只能处理 32 位范围的值，超出时需要 `movabsq`）。

```
cmp b, a
```

执行 `a-b` 但不保存结果，只是根据结果更新标志位寄存器。

标志位	名称	含义
ZF	Zero Flag	若 <code>a == b</code> ，则设置为 1，否则为 0
SF	Sign Flag	若 <code>a - b</code> 的结果为负（最高位为1），则设置为 1
OF	Overflow Flag	若有符号运算中发生溢出（即正数减负数结果为负，或负数减正数为正）
CF	Carry Flag	用于无符号运算中判断是否发生借位（即 <code>a < b</code> ）

例：假设有这样的代码

```
mov eax, 3 ; a = 3
mov ebx, 5 ; b = 5
cmp ebx, eax ; cmp b, a → 3 - 5
```

`3 - 5` 会产生**借位**，所以 `CF = 1`

所以**判断 `a < b` 时，使用 `jb` (jump below)**

`3 - 5 = -2` → `SF = 1`，`ZF = 0`，`OF = 0`

所以 判断 `a < b` 时, 使用 `j1` (`jump less`)

条件跳转指令对应表:

C 比较语句	对应汇编顺序	汇编条件跳转指令	有符号/无符号
<code>if (a == b)</code>	<code>cmp b, a</code>	<code>je / jz</code>	通用
<code>if (a != b)</code>	<code>cmp b, a</code>	<code>jne / jnz</code>	通用
<code>if (a > b)</code>	<code>cmp b, a</code>	<code>jg</code>	有符号
<code>if (a < b)</code>	<code>cmp b, a</code>	<code>j1</code>	有符号
<code>if (a >= b)</code>	<code>cmp b, a</code>	<code>jge</code>	有符号
<code>if (a <= b)</code>	<code>cmp b, a</code>	<code>jle</code>	有符号
<code>if (a > b)</code>	<code>cmp b, a</code>	<code>ja</code>	无符号
<code>if (a < b)</code>	<code>cmp b, a</code>	<code>jb</code>	无符号
<code>if (a >= b)</code>	<code>cmp b, a</code>	<code>jae</code>	无符号
<code>if (a <= b)</code>	<code>cmp b, a</code>	<code>jbe</code>	无符号

例:

C 代码:

```
if (x < y) {  
    // do something  
}
```

汇编:

```
mov eax, x  
mov ebx, y  
cmp ebx, eax    ; = x - y  
j1 label1       ; 因为是有符号比较
```

条件传输

```
cmovcc src, dst
```

不支持单字节传输

常见cc代码:

条件码 cc	含义	等价条件 (针对 <code>cmp b, a</code>)	是否有符号
<code>e / z</code>	equal / zero	<code>a == b</code>	否
<code>ne / nz</code>	not equal / not zero	<code>a != b</code>	否

条件码 cc	含义	等价条件 (针对 <code>cmp b, a</code>)	是否有符号
g	greater	<code>a > b</code>	有符号
ge	greater or equal	<code>a >= b</code>	有符号
l	less	<code>a < b</code>	有符号
le	less or equal	<code>a <= b</code>	有符号
a	above	<code>a > b</code>	无符号
ae	above or equal	<code>a >= b</code>	无符号
b	below	<code>a < b</code>	无符号
be	below or equal	<code>a <= b</code>	无符号
s	sign	结果为负	-
ns	not sign	结果为非负	-

`cmov` 系列是寄存器到寄存器传输。

扩展传输：

符号扩展从一字节到四字节

```
movsbl S, D
```

无符号扩展从一字节到四字节

```
movzbl S, D
```

x86/64有一个逆天属性，`movl` 的目的地是寄存器的时候，会把高四字节设置成 0

内存寻址：

`D(Rb, Ri, S)` 对应 `M[R[Rb]+R[Ri]*S+D]`

入栈：

```
pushq S
```

意义：`%rsp = %rsp - 8`（栈向低地址增长）`(%rsp) = S`

出栈：

```
popq D
```

意义：`D = (%rsp)` `%rsp = %rsp + 8`

压栈要字对齐，所以一次必须压64位进去（64位机器）

```
leaq S, d
```

`leaq 8(%rax, %rbx, 4), %rcx`做的事情是 $rcx = rax + rbx * 4 + 8$

算术指令：

加法和减法

指令	作用	说明
<code>add</code>	加法	$D = D + S$
<code>inc</code>	自增1	给操作数加1
<code>sub</code>	减法	$D = D - S$
<code>dec</code>	自减1	给操作数减1
<code>neg</code>	取负	取操作数的相反数的补码

乘法和除法

指令	作用	说明
<code>mul</code>	无符号乘法	用无符号数乘，结果存储在指定寄存器对中
<code>imul</code>	有符号乘法	$D = D * S$
<code>div</code>	无符号除法	除法，结果商和余数分别放在寄存器中
<code>idiv</code>	有符号除法	有符号除法，支持负数

逻辑和位操作（虽然不纯算术，但常一起用）

指令	作用
<code>and</code>	位与
<code>or</code>	位或
<code>xor</code>	位异或
<code>not</code>	位取反

其它算术相关指令

指令	作用
<code>sar</code>	算术右移（带符号扩展） $D = D \gg S$
<code>shr</code>	逻辑右移（零填充） $D = D \gg S$
<code>sal / shl</code>	左移 $D = D \ll S$

指令	效果	描述
<code>imulq S</code>	$R[\%rdx]: R[\%rax] \leftarrow S \times R[\%rax]$	有符号全乘法
<code>mulq S</code>	$R[\%rdx]: R[\%rax] \leftarrow S \times R[\%rax]$	无符号全乘法
<code>cltq</code>	$R[\%rdx]: R[\%rax] \leftarrow \text{符号扩展}(R[\%rax])$	转换为八字
<code>idivq S</code>	$R[\%rdx] \leftarrow R[\%rdx]: R[\%rax] \bmod S$ $R[\%rdx] \leftarrow R[\%rdx]: R[\%rax] \div S$	有符号除法
<code>divq S</code>	$R[\%rdx] \leftarrow R[\%rdx]: R[\%rax] \bmod S$ $R[\%rdx] \leftarrow R[\%rdx]: R[\%rax] \div S$	无符号除法

图 3-12 特殊的算术操作。这些操作提供了有符号和无符号数的全 128 位乘法和除法。一对寄存器 `%rdx` 和 `%rax` 组成一个 128 位的八字

`div` 指令是 **x86** 汇编中的**无符号除法指令**，它的行为和操作数大小密切相关，具体定义如下：

对于 64 位除法 (`div r/m64`, **x86-64**)

被除数是 **128 位** 寄存器 `%rdx:%rax` (`%rdx` 高 64 位, `%rax` 低 64 位)

除数是 64 位寄存器或内存操作数

结果：商放在 `%rax` 余数放在 `%rdx`

注意点: **商溢出** (商大于目标寄存器大小) 时, CPU 会触发异常 (`#DE`, 除法错误), **除数为 0** 也会触发异常

Question:

```
fun:
    push %rbp
    mov %rsp, %rbp
    movl $0x0, -0x4(%rbp)
    ...
    leave
    ret
```

这几句话到底在干什么？

Answer:

这几句话是典型的函数开头与返回，用于在函数调用时**建立栈帧**，方便局部变量管理和参数访问。我们不妨看看整个函数调用返回过程。我们在一段指令中间有一个 `call`

```
---
call fun
---
```

假设调用函数之前，栈顶位置为 `%rsp=0x00000F00`

```
0x00000F00 <- %rsp
```

`call` 时，假设调用函数的地址是 `0x00001000`

`call` 使得 `%rip = 0x00001000`

call 之后的语句的位置是 0x00100000

则压栈

```
0x00000EF8 <-%rsp, (%rsp)=0x00100000
```

然后读取程序计数器下一条指令地址 0x00001000 进入被调用函数

pushq %rbp 使得

```
0x00000EF0 <-%rsp, (%rsp)=调用者栈帧
```

movq %rsp, %rbp 更新栈帧，意思就是被调用函数从这里开始

```
0x00000EF0 <-%rsp,%rbp
```

被调用函数执行完任务后执行 leave，也就是

```
mov %rbp, %rsp
```

使得栈顶回收，销毁了栈中的临时变量。

```
0x00000EF0 <-%rsp,%rbp
```

还有

```
pop %rbp
```

使得

```
%rbp=(%rsp)=调用者栈帧
```

```
0x00000EF8 <-%rsp, (%rsp)=0x00100000
```

再执行 ret 使得

```
%rip=(%rsp)
```

```
0x00000F00 <-%rsp
```

栈顶和栈帧都回到了调用前的位置，程序计数器也得到了下一条指令的地址，接下去就会执行调用的下一行指令。

算术指令会对条件码隐含赋值

CF (Carry Flag) 进位标志

表示**无符号**加法产生了进位，或者**无符号**减法发生了借位。

加法：若结果 **超出目标数据类型的最大值**（即发生无符号溢出）

减法：若被减数 < 减数，即**借位**

示例：

```
movb $0xFF, %al    ; 255
addb $1, %al        ; 255 + 1 = 0 (溢出, CF=1)
```

ZF (Zero Flag) 零标志

当操作结果为 0 时，ZF = 1

加法、减法、与 (AND)、异或 (XOR) 等操作结果为 0 → ZF 被置 1

示例：

```
movl $5, %eax
subl $5, %eax        ; 5 - 5 = 0 → ZF = 1
```

SF (Sign Flag) 符号标志

反映结果的最高位 (符号位)，用于有符号数判断正负

若结果为负数 (即最高位是 1) → SF = 1

若结果为正数或 0 (即最高位是 0) → SF = 0

示例：

```
movl $5, %eax
subl $10, %eax       ; 5 - 10 = -5 → SF = 1
```

OF (Overflow Flag) 溢出标志

表示有符号数加/减运算时，结果超出了可表示的范围 (溢出)

加法中：两个同号数相加，结果为异号 → OF = 1

减法中：被减数与减数符号不同，结果符号与被减数不同 → OF = 1

示例 (加法)：

```
movb $127, %al      ; 最大正数
addb $1, %al         ; 127 + 1 = -128 → OF = 1 (溢出)
```

示例 (减法)：

```
movb $-128, %al     ; 最小负数
subb $1, %al         ; -128 - 1 = 127 (结果是正数) → OF = 1
```

setX D 用于根据条件 X 把条件码复制到 D 的低字节。

示例

```
int gt(long x, long y)
{return x > y;}
```

```
cmpq %rsi, %rdi
setg %al
movzbl %al, %eax
ret
```

循环语句在汇编中的经典形式：

```
int i = 0;
while (i < 10) {
    // do something
    i++;
}
```

对应汇编 (x86-64, 假设 `i` 在 `%eax`)：

```
movl $0, %eax          ; i = 0
loop_start:
    cmpl $10, %eax      ; compare i and 10
    jge loop_end        ; if i >= 10, jump to end
    ; do something
    incl %eax           ; i++
    jmp loop_start      ; repeat
loop_end:
```

```
for (int i = 0; i < 10; i++) {
    // do something
}
```

对应汇编：

```
movl $0, %eax
for_loop:
    cmpl $10, %eax
    jge for_end
    ; do something
    incl %eax
    jmp for_loop
for_end:
```

```
int i = 0;
do {
    // do something
    i++;
} while (i < 10);
```

对应汇编：

```

movl $0, %eax
do_loop:
    ; do something
    incl %eax
    cmpl $10, %eax
    jl do_loop

```

switch语句:

switch 使用一个叫做跳转表的东西来实现。跳转表是一个数组，表项 `i` 是一个代码段的地址，这个代码段实现当开关索引值等于 `i` 时程序应该采取的动作。

举个例子:

```

void switch_eg(long x, long n, long *dest) {
    long val = x;

    switch (n) {
        case 100:
            val *= 13;
            break;
        case 102:
            val += 10;
            // Fall through
        case 103:
            val += 11;
            break;
        case 104:
        case 106:
            val *= val;
            break;
        default:
            val = 0;
    }

    *dest = val;
}

```

对应的汇编:

```

# void switch_eg(long x, long n, long *dest)
# 参数: x -> %rdi, n -> %rsi, dest -> %rdx
# 回顾一下每个寄存器的一般用途, 比如说%rdi一般就是存放第一个参数的

switch_eg:
    subq $100, %rsi          # index = n - 100
    cmpq $6, %rsi           # 比较 index 是否 > 6
    ja .L8                  # 如果 index > 6, 跳转到 default 分支

    # 使用跳转表: 跳转到 L4 + index * 8 (每个地址占8字节)
    jmp *.L4(, %rsi, 8)      # 跳转到对应 case 标签

.L3:                       # case 100:

```

```

    leaq (%rdi, %rdi, 2), %rax # rax = x + x*2 = 3*x
    leaq (%rdi, %rax, 4), %rdi # val = 13*x
    jmp .L2                    # 跳转到 done

.L5:                          # case 102:
    addq $10, %rdi            # x += 10
    # fall through

.L6:                          # case 103:
    addq $11, %rdi            # x += 11
    jmp .L2                  # 跳转到 done

.L7:                          # case 104, 106:
    imulq %rdi, %rdi          # val = x * x
    jmp .L2                  # 跳转到 done

.L8:                          # default:
    movl $0, %edi             # val = 0

.L2:                          # done:
    movq %rdi, (%rdx)         # *dest = val
    ret                      # 返回

# 跳转表
.section .rodata
# 存储于只读数据段中
.align 8                    # align address to multiple of 8
.L4:
    .quad .L3                # case 100
    .quad .L8                # case 101 -> default
    .quad .L5                # case 102
    .quad .L6                # case 103
    .quad .L7                # case 104
    .quad .L8                # case 105 -> default
    .quad .L7                # case 106

```

Question:

跳转表的这种星号是什么意思？`jmp *.L4(, %rsi, 8)`

Answer:

这是一个非常经典的 **跳转表间接跳转**用法，`jmp *.L4(, %rsi, 8)` 中的星号 `*` 代表的是“间接跳转”。

`jmp *.L4(, %rsi, 8)` 的含义是跳转到位于内存地址 `.L4 + %rsi * 8` 所指向的地址处

也就是 `.L4` 是跳转表的基地址，`%rsi` 是我们已经计算好的 `index = n - 100`，`*` 表示“间接跳转”，即，它不是跳到 `.L4 + %rsi*8` 这个地址本身，而是 **跳到这个地址中存储的 8 字节的值所指的地址**（即代码标签地址）。

Question:

这段代码中把 `%rdx` 复制到 `%rbx` 是不是多此一举？

```
push %rbx
mov %rdx,%rbx
callq 400550 <mult2>
mov %rax, (%rbx)
pop %rbx
retq
```

在Linux/x86-64 下的函数调用规范中，`%rdx` 是 **caller-saved** 寄存器（调用者保存），也就是说，**被调用函数可以随便改 `%rdx`，不会负责还回来。**

因此如果你要在 `call` 之后仍然使用 `%rdx`，你就得自己提前保存它。

所以为什么用 `%rbx`？`%rbx` 是 **callee-saved** 的，被调用函数要保证它不变；调用者用它当“临时保存空间”很安心。

加上**caller-saved**或**callee-saved**属性，可以再复习一遍寄存器用途：

寄存器	用途分类	说明
<code>%rax</code>	返回值寄存器	函数返回值放在这里。也常用于中间计算。
<code>%rbx</code>	被调用者保存（callee-saved）	函数必须在使用前保存它，并在返回前恢复。用于保存重要值。
<code>%rcx</code>	参数寄存器	调用函数时传递第 4 个参数。caller-saved。
<code>%rdx</code>	参数寄存器	调用函数时传递第 3 个参数。caller-saved。
<code>%rsi</code>	参数寄存器	调用函数时传递第 2 个参数。caller-saved。
<code>%rdi</code>	参数寄存器	调用函数时传递第 1 个参数。caller-saved。
<code>%rsp</code>	栈指针	永远指向当前栈顶， 压栈/出栈操作都依赖它。
<code>%rbp</code>	栈帧基址	用作当前函数栈帧的基准点。也可作为通用寄存器。
<code>%r8 ~ %r9</code>	参数寄存器	调用函数时传递第 5、6 个参数。caller-saved。
<code>%r10, %r11</code>	scratch 临时寄存器	caller-saved，短期计算/中转用，不能指望调用完还在。
<code>%r12 ~ %r15</code>	被调用者保存	callee-saved。用于长生命周期的变量或跨函数保存数据。

多出来的参数会放在栈中。

实例：调用者保存（Caller-saved）

```
movq %rax, -8(%rsp) ; 保存 %rax
callq some_func
movq -8(%rsp), %rax ; 恢复 %rax
```

因为 `%rax` 是 caller-saved，被调用函数可能会改掉它。

实例：被调用者保存（Callee-saved）

```
some_func:
    pushq %rbx        ; 保存 %rbx
    ...              ; 使用 %rbx 做一些事
    popq %rbx         ; 恢复 %rbx
    ret
```

因为 `%rbx` 是 callee-saved，使用它前必须保存。

递归函数：

```
pcount_r:
    movl $0, %eax
    testq %rdi, %rdi
    je .L6
    pushq %rbx
    movq %rdi, %rbx
    andl $1, %ebx
    shrq %rdi
    call pcount_r
    addq %rbx, %rax
    popq %rbx
.L6:
    rep; ret
```

Question:

上面为什么是调用者 `pushq %rbx` 保存 `%rbx`，不是 callee-saved 嘛？

Answer:

虽然 `%rbx` 是 callee-saved，但这段代码里是函数自己在递归调用自己，也就是说，它既是 caller，也是 callee。

所以，它在调用自己之前用到了 `%rbx`，就必须手动保存自己的 `%rbx`，否则下一次递归会覆盖掉它的值。

```
pcount_r:
    movl $0, %eax        ; 初始化返回值为 0
    testq %rdi, %rdi     ; 判断参数 rdi 是否为 0
    je .L6               ; 如果是 0，跳转到返回语句
    pushq %rbx           ; 保存 %rbx
    movq %rdi, %rbx      ; 将 rdi 赋值到 rbx（暂存原始值）
    andl $1, %ebx        ; 得到最低位 (x & 1)
    shrq %rdi            ; rdi >>= 1
    call pcount_r        ; 递归调用自己
    addq %rbx, %rax       ; 把低位加到结果上
    popq %rbx            ; 恢复 %rbx
.L6:
    rep; ret
```

如果不保存：

```

pcount_r:
    movl $0, %eax
    testq %rdi, %rdi
    je .L6
    movq %rdi, %rbx
    andl $1, %ebx
    shrq %rdi
    call pcount_r
    addq %rbx, %rax
.L6:
    rep; ret

```

这样会导致 `%rbx` 被 callee 篡改，一直是最后一个 `%rbx`。

栈的大小由操作系统规定，**linux** 为 **8M**。

结构体与对齐：

每个成员的偏移地址必须是它自身大小（或指定对齐值）的整数倍；

结构体总大小必须是其最大成员对齐值的倍数；

结构体整体的起始地址必须满足其最大成员的对齐要求（整数倍）。

为节省空间，我们在设计结构体的时候会把大尺寸数据放在结构体前面。

我们把变量分成 3 大类来看：

1. 栈上分配

函数内部的局部变量、作用域结束后自动释放。

```

void foo() {
    int a = 10;           // a 在栈上
    char buf[100];        // buf 是栈上数组
}

```

空间较小（一般 **8MB** 左右）

不能返回指针！因为函数结束之后栈就回收了 `%rsp=%rbp`

2. 堆上分配

`malloc/new` 出来的，自己控制生命周期。

```

char *p = malloc(100);    // p 指向堆上 100 字节
int *a = new int[10];     // C++ 的堆分配

```

空间大（几 **GB** 甚至 **TB**）

掌握主动权（用 `free` 或 `delete`）

3. 数据段（`.data` / `.bss`）分配

全局变量、静态变量会被放在 `.data`（已初始化）或 `.bss`（未初始化）段。

`.data` 段（**有初始值**）


```
int g = 42;           // g 在 .data 段
static float pi = 3.14; // pi 也在 .data 段
```

`.bss` 段 (未初始化, 全 0 清空)

```
static int counter; // counter 在 .bss 段, 值默认为 0
int arr[10000];     // 如果是全局定义也在 .bss
```

在程序启动时加载进内存, 生命周期从程序开始到结束, 不随函数调用进出。

缓冲区溢出攻击:

防护手段:

- 1.避免溢出漏洞
- 2.随机栈偏移
- 3.将栈标记为不可执行
- 4.栈金丝雀

Question:

这段代码怎么理解:

```
mov %fs:0x28, %rax
mov %rax, 0x8(%rsp)
```

Answer:

这是经典金丝雀保存代码。在 Linux x86_64 系统中, `FS:0x28` 是线程局部存储中保存金丝雀值的地址。每个线程有自己独立的 `FS` 段, 因此金丝雀是线程局部的。第一条指令读取金丝雀值到 `%rax`

第二条指令把 `%rax` 中刚刚读到的金丝雀值, 保存到栈上的偏移地址 `0x8(%rsp)`。也就是**当前函数栈帧中的金丝雀存放位置**。

检测:

```
mov 0x8(%rsp), %rax
xor %fs:0x28, %rax
```

攻击随机栈偏移可以用 `nop sled`。

例题:

- 1.简述缓冲区溢出攻击的原理以及防范方法(5分)。

攻击原理(3个采分点):

- (1) 程序输入缓冲区写入特定的数据, 例如在 `gets` 读入字符串时
- (2) 使位于栈中的缓冲区数据溢出, 用特定的内容覆盖栈中的内容, 例如函数返回地址等
- (3) 使得程序在读入字符串, 结束函数 `gets` 从栈中读取返回地址时, 错误地返回到特定的位置, 执行特定的代码, 达到攻击的目的。

防范方法(2个采分点):

(1) 代码中避免溢出漏洞: 例如使用限制字符串长度的库函数。

(2) 随机栈偏移: 程序启动后, 在栈中分配随机数量的空间, 将移动整个程序使用的栈空间地址。

(3) 限制可执行代码的区域

(4) 进行栈破坏检查--金丝雀

补充解释: 对抗缓冲区溢出攻击4种方法

(1) 使用限制字符串长度的库函数

(2) 栈随机化

栈随机化的思想使得栈的位置在程序每次运行时都有变化。程序开始时, 在栈上分配一段 $0 \sim n$ 字节之间的随机大小的空间。分配的范围 n 必须足够大, 才能获得足够多的栈地址变化, 但是又要足够小, 不至于浪费程序太多空间。

Linux系统中, 栈随机化已经变成了标准行为。是更大的一类技术中的一种, 这类技术称为地址空间布局随机化。每次运行时程序的不同部分, 包括程序代码、库代码、栈、全局变量和堆数据, 都会被加载到内存的不同区域。

(3) 限制可执行代码区域

消除攻击者向系统中插入可执行代码的能力, 限制哪些内存区域能够存放可执行代码, 只有保存编译器产生的代码的那部分才需要是可执行的, 其他部分可以被限制为只允许读和写。

(4) 栈破坏检测

最近的 GCC 版本在产生的代码中加入了一种栈保护者机制, 来检测缓冲区越界。其思想是在栈帧中任何局部缓冲区与栈状态之间存储一个特殊的值, 也称哨兵值, 是在程序每次运行时随机产生的。在恢复寄存器状态和从函数返回前, 程序检查这个金丝雀值是否被该函数的某个操作或者该函数调用的某个函数的某个操作改变了。

后面就不是我的活了

指令集: CISC复杂指令集, 代表是x86

用栈传参数, 保存程序计数器

RSIC

Y86-64:

字节	0	1	2	3	4	5	6	7	8	9
halt	0	0								
nop	1	0								
rrmovq rA, rB	2	0	rA	rB						
irmovq V, rB	3	0	F	rB					V	
rmmovq rA, D(rB)	4	0	rA	rB					D	
mrmmovq D(rB), rA	5	0	rA	rB					D	
OPq rA, rB	6	fn	rA	rB						
jXX Dest	7	fn							Dest	
cmovXX rA, rB	2	fn	rA	rB						
call Dest	8	0							Dest	
ret	9	0								
pushq rA	A	0	rA	F						
popq rA	B	0	rA	F						

寄存器文件中包括PC、CC、Stat和%rax, %rsp等

寄存器文件读是组合逻辑，写是时序逻辑，只在时钟上升沿更新

顺序执行六个阶段

取指，译码，执行，访存，写回，PC更新

取指(fetch): 取指阶段从内存读取指令字节，地址为程序计数器(PC)的值。从指令中抽取出指令指示符字节的两个四位部分，称为 icode(指令代码) 和 ifun(指令功能)。它可能取出一个寄存器指示符字节，指明一个或两个寄存器操作数指示符 rA 和 rB。它还可能取出一个四字节常数字 valc。它按顺序方式计算当前指令的下一条指令的地址 valP。也就是说，valP 等于 PC 的值加上已取出指令的长度。

译码(decode): 译码阶段从寄存器文件读入最多两个操作数，得到值 valA 和 / 或 valB, 通常，它读入指令 rA 和 rB 字段指明的寄存器，不过有些指令是读寄存器 %rsp 的。

执行(execute): 在执行阶段，算术 / 逻辑单元(ALU)要么执行指令指明的操作（根据 ifun 的值），计算内存引用的有效地址，要么增加或减少栈指针。得到的值我们称为 valE。在此，也可能设置条件码。对一条条件传送指令来说，这个阶段会检验条件码和传送条件（由 ifun 给出），如果条件成立，则更新目标寄存器。同样，对一条跳转指令来说，这个阶段会决定是不是应该选择分支。

访存(memory): 访存阶段可以将数据写入内存，或者从内存读出数据。读出的值为 valM。

写回(write back): 写回阶段最多可以写两个结果到寄存器文件。

更新 PC(PC update): 将 PC 设置成下一条指令的地址。

阶段	OPq rA, rB	rrmovq rA, rB	irmovq V, rB
取指	icode: ifun $\leftarrow M_1[PC]$ rA: rB $\leftarrow M_1[PC+1]$ valP $\leftarrow PC+2$	icode: ifun $\leftarrow M_1[PC]$ rA: rB $\leftarrow M_1[PC+1]$ valP $\leftarrow PC+2$	icode: ifun $\leftarrow M_1[PC]$ rA: rB $\leftarrow M_1[PC+1]$ valC $\leftarrow M_8[PC+2]$ valP $\leftarrow PC+10$
译码	valA $\leftarrow R[rA]$ valB $\leftarrow R[rB]$	valA $\leftarrow R[rA]$	
执行	valE $\leftarrow valB \text{ OP } valA$ Set CC	valE $\leftarrow 0 + valA$	valE $\leftarrow 0 + valC$
访存			
写回	R[rB] $\leftarrow valE$	R[rB] $\leftarrow valE$	R[rB] $\leftarrow valE$
更新 PC	PC $\leftarrow valP$	PC $\leftarrow valP$	PC $\leftarrow valP$

阶段	rmmovq rA, D(rB)	mrmmovq D(rB), rA
取指	icode: ifun $\leftarrow M_1[PC]$ rA: rB $\leftarrow M_1[PC+1]$ valC $\leftarrow M_8[PC+2]$ valP $\leftarrow PC+10$	icode: ifun $\leftarrow M_1[PC]$ rA: rB $\leftarrow M_1[PC+1]$ valC $\leftarrow M_8[PC+2]$ valP $\leftarrow PC+10$
译码	valA $\leftarrow R[rA]$ valB $\leftarrow R[rB]$	valB $\leftarrow R[rB]$
执行	valE $\leftarrow valB + valC$	valE $\leftarrow valB + valC$
访存	M ₈ [valE] $\leftarrow valA$	valE $\leftarrow M_8[valE]$
写回		
		R[rA] $\leftarrow valM$
更新 PC	PC $\leftarrow valP$	PC $\leftarrow valP$

阶段	pushq rA	popq rA
取指	icode: ifun $\leftarrow M_1[PC]$ rA: rB $\leftarrow M_1[PC+1]$ valP $\leftarrow PC+2$	icode: ifun $\leftarrow M_1[PC]$ rA: rB $\leftarrow M_1[PC+1]$ valP $\leftarrow PC+2$
译码	valA $\leftarrow R[rA]$ valB $\leftarrow R[\%rsp]$	valA $\leftarrow R[\%rsp]$ valB $\leftarrow R[\%rsp]$
执行	valE $\leftarrow valB - 8$	valE $\leftarrow valB + 8$
访存	M ₈ [valE] $\leftarrow valA$	valM $\leftarrow M_8[valA]$

阶段	pushq rA	popq rA
写回	$R[\%rsp] \leftarrow valE$	$R[\%rsp] \leftarrow valE$ $R[rA] \leftarrow valM$
PC更新	$PC \leftarrow valP$	$PC \leftarrow valP$

popq 指令的执行与 pushq 的执行类似，除了在译码阶段要读两次栈指针以外。这样做看上去很多余，但是我们会看到让 valA 和 valB 都存放栈指针的值，会使后面的流程跟其他的指令更相似，增强设计的整体一致性。

阶段	jxx Dest	call Dest	ret
取指	$icode, ifun \leftarrow M_1[PC]$ $valC \leftarrow M_8[PC+1]$ $valP \leftarrow PC+9$	$icode, ifun \leftarrow M_1[PC]$ $valC \leftarrow M_8[PC+1]$ $valP \leftarrow PC+9$	$icode, ifun \leftarrow M_1[PC]$ $valP \leftarrow PC+1$
译码		$valB \leftarrow R[\%rsp]$	$valA \leftarrow R[\%rsp]$ $valB \leftarrow R[\%rsp]$
执行	$Cnd \leftarrow Cond(CC, ifun)$	$valE \leftarrow valB + (-8)$	$valE \leftarrow valB + 8$
访存		$M_8[valE] \leftarrow valP$	$valM \leftarrow M_8[valA]$
写回		$R[\%rsp] \leftarrow valE$	$R[\%rsp] \leftarrow valE$
更新 PC	$PC \leftarrow Cnd ? valC : valP$	$PC \leftarrow valC$	$PC \leftarrow valM$